



Univerzitet u Nišu
Elektronski fakultet u Nišu



Turso - distribuirana baza podataka za cloud i edge

Seminarski rad

Mentor:
Prof. dr Aleksandar Stanimirović

Student:
Luka Veličković, 2101

Niš, Jul 2026.

Sadržaj

1 Uvod.....	3
2 SQLite, libSQL, Turso.....	4
2.1 SQLite.....	4
2.1.1 SQLite file.....	4
2.1.2 Izolacija kod SQLite-a.....	6
2.1.3 Funkcionalnosti SQLite-a.....	6
2.1.4 SQLite use-cases.....	7
2.2 libSQL.....	7
2.3 Turso.....	8
2.3.1 CDC.....	8
2.3.2 Konkurentni upis.....	9
2.3.3 Sync.....	10
3 Turso Cloud.....	12
3.1 Cloud specifikacija.....	12
3.2 Embedded replicas.....	14
3.3 Branching.....	15
3.4 Point-in-Time recovery.....	15
4 Praktični prikaz.....	16
4.1 Turso database.....	16
4.1.1 CDC.....	17
4.1.2 Konkurentni upisi.....	19
4.1.3 Sync.....	21
4.2 Turso cloud.....	23
4.2.1 Embedded replicas.....	25
4.2.2 Branching.....	28
4.2.3 Point-in-time recovery.....	28
5 Literatura.....	31

1 Uvod

Tradicionalne baze podataka najčešće su projektovane tako da se centralna instanca baze nalazi na jednom serveru ili u jednom regionu, dok aplikacioni serveri komuniciraju sa njom preko mreže. Ovakav pristup je jednostavan za razumevanje i upravljanje, ali može postati ograničavajući u sistemima koji zahtevaju veoma malu latenciju, visoku dostupnost i geografski distribuiran pristup podacima. Ukoliko se korisnik nalazi daleko od regiona u kome je baza podataka pokrenuta, svaka operacija čitanja ili upisa može zahtevati mrežnu komunikaciju preko velikih geografskih udaljenosti. Zbog toga se u savremenim cloud i edge okruženjima sve češće javlja potreba za bazama podataka koje mogu da približe podatke korisnicima, a da pritom zadrže jednostavnost korišćenja karakterističnu za klasične relacione baze.

SQLite predstavlja jednu od najrasprostranjenijih relacionih baza podataka, pre svega zbog svoje jednostavnosti, pouzdanosti i činjenice da se cela baza najčešće nalazi u jednom fajlu. Za razliku od klijent-server sistema, SQLite se izvršava unutar same aplikacije, bez potrebe za posebnim serverskim procesom. Ovakav pristup čini SQLite izuzetno pogodnim za lokalno skladištenje podataka, embedded sisteme, mobilne aplikacije i druge scenarije u kojima je poželjno imati malu, brzu i samostalnu bazu podataka. Ipak, SQLite po svojoj osnovnoj arhitekturi nije projektovan kao distribuirana baza podataka. Njegova jednostavnost i lokalni karakter predstavljaju prednost u mnogim slučajevima, ali mogu biti ograničenje kada je potrebno obezbediti globalnu distribuciju, replikaciju, sinhronizaciju i rad u cloud okruženju.

Turso nastaje kao pokušaj da se osobine SQLite-a iskoriste u modernom distribuiranom okruženju. Turso pokušava da spoji prednosti embedded baze podataka sa zahtevima modernih aplikacija koje se izvršavaju u globalno distribuiranom okruženju.

Posebno značajan aspekt Turso baze podataka jeste njena orijentacija ka edge computing modelu. Edge computing podrazumeva izvršavanje aplikacionih komponenti što bliže korisniku, umesto isključivo u centralizovanom data centru. Kada se aplikacioni kod izvršava na edge lokacijama, baza podataka može postati usko grlo ukoliko i dalje ostaje centralizovana u jednom regionu. Zbog toga Turso uvodi koncepte kao što su embedded replicas, sinhronizacija između lokalnih i udaljenih instanci, branching i point-in-time recovery. Ove funkcionalnosti omogućavaju da se podaci približe aplikaciji i korisniku, ali i da se zadrže mehanizmi za oporavak, razvoj, testiranje i upravljanje verzijama podataka.

Ovaj rad se fokusira na proučavanje Turso baze podataka kao distribuiranog rešenja zasnovanog na SQLite/libSQL ekosistemu. U radu se najpre razmatraju osnovne karakteristike SQLite-a, njegova fajl struktura, izolacija, funkcionalnosti i tipični slučajevi upotrebe. Nakon toga se uvodi libSQL kao proširenje SQLite-a i osnova za Turso, uz poseban osvrt na mehanizme kao što su change data capture, konkurentni upisi i sinhronizacija. U nastavku se analizira Turso Cloud, uključujući njegovu cloud specifikaciju, embedded replike, branching i point-in-time recovery. Pored teorijskog pregleda, rad sadrži i praktični prikaz kroz konkretne primere rada sa Turso bazom podataka i Turso Cloud platformom.

Cilj rada je da prikaže na koji način Turso proširuje tradicionalni model SQLite baze podataka i prilagođava ga potrebama cloud i edge aplikacija. Kroz analizu arhitekture, funkcionalnosti i praktičnih primera, rad nastoji da objasni koje probleme Turso rešava, koje prednosti donosi u odnosu na klasičan pristup korišćenju SQLite-a, ali i koja ograničenja i kompromisi postoje prilikom korišćenja distribuirane baze podataka zasnovane na ovakvom modelu.

2 SQLite, libSQL, Turso

2.1 SQLite

SQLite je biblioteka koja implementira *self-contained, serverless, zero-configuration* i *transactional SQL database engine*.

- *Self-contained* – osobina koja znači da SQLite ima veoma malo broj zavisnosti. Može da se izvršava na bilo kom operativnom sistemu, čak i na bare-bones embedded operativnim sistemima. SQLite nema eksterne zavisnosti, osim par osnovnih C sistemskih poziva. Cela SQLite biblioteka može da se enkapsulira u jedan source fajl i ne zahteva posebne build sisteme. [1]
- *Serverless* – SQLite je klasična serverless baza podataka, u smislu da se database engine izvršava u istom procesu ili threadu kao i sama aplikacija. Kod većina modernih baza podataka, DBMS se izvršava kao poseban proces, najčešće na odvojenoj mašini, a komunikacija sa bazom se odvija preko TCP/IP protokola. Ovakav pristup ima prednosti, najviše u pogledu jednostavnosti korišćenja, konfiguracije i otkrivanja problema. Nedostaci serverless pristupa uključuju nivo kontrole koje baza podataka ima, u vidu granularnosti zaključavanja i konkurentnog pristupa. U ovom kontekstu serverless ne treba mešati sa istim terminom koji se koristi za opisivanje modela cloud execution modela.
- *Zero-configuration* – SQLite nema proces instalacije ili podešavanja. Nema potrebe za administratorom koji kreira novu instancu baze ili dodeljivanjem permisija. Nema potrebe za specijalnim postupkom oporavka nakon pada sistema. [1]
- *Transactional* - SQLite nudi ACID svojstva. Implementira serializable nivo izolacije.

Radi se o open-code projektu, međutim projekat je zatvoren za kontribucije. Projekat održavaju tri developera, a SQLite je najkorišćenija baza podataka na svetu. Poznata je činjenica da SQLite ima jedan od najopširnijih skupova testova, sa 100% pokrivenošću grana. [1]

Kao što je napomenuto, SQLite se ne izvršava u posebnom procesu pa je bolji način da se o SQLite-u razmišlja, ne kao zameni za Oracle ili Postgres, već kao zameni za *foopen*. Cela SQL baza sa većim brojem tabela, indeksa, trigera i pogleda se skladišti na jednom fajlu na disku. Format fajla je cross-platform i može bezbedno da se kopira između različitih arhitektura pa se vrlo često koristi kao application file format, odnosno format u koji se skladišti stanje aplikacije. [1]

2.1.1 SQLite file

Kako SQLite sav sadržaj baze skladišti u jednom fajlu, struktura ovog fajla je jako složena. Tokom izvršavanja transakcije, SQLite koristi i drugi fajl „rollback journal“ ili wal fajl u kome se skladište dodatne informacije.

Prvih 100 bajtova je header baze podataka, koji sadrži podatke kao što je verzija SQLite-a, veličina stranice, veličina page keša, encoding koji se koristi, broj slobodnih stranica, broj prve freelist stranice itd.

SQLite, kao i većina drugih relacionih DBMS-ova, organizuje podatke u stranice. Veličina stranice je stepen dvojke između 512 i 2^{16} . Sve stranice su iste veličine. Postoji nekoliko tipova stranica:

- b-tree stranica
 - interna b-tree stranica tabele
 - list b-tree stranica tabele
 - interna b-tree stranica indeksa
 - list b-tree stranica indeksa
- freelist stranica
 - trunk freelist stranica
 - list freelist stranica
- lock-byte stranica
- pointer map stranica
- payload overflow stranica

Lock byte stranica se koristi kod SQLite baza koje su veće od 1GB i takve baze imaju tačno jednu lock byte stranicu. Baze manje od 1GB nemaju lock byte stranicu. SQLite core ne modifikuje, niti čita iz ove stranice. Stranica postoji zbog backward compatibility-ja, prvenstveno zbog potrebe da se podrži Windows 95. Stranica služi SQLite VFS-u, obzirom da Windows 95 implementira isključivo mandatory file locking. [1]

Freelist stranice se koriste kako bi se pratile stranice koje više nisu u upotrebi (npr. posledica brisanja podataka iz baze). Ovakve stranice (preciznije pokazivači do njih) se skladište u freelist stranicama. Freelist stranice su organizovane kao lančana lista. [1]

B-tree stranice se koriste za skladištenje korisničkih podataka – tabela i indeksa. Svako B stablo sadrži dva tipa stranica interne stranice i listove. Unutar interne stranice skladište se interni čvorovi B stabla, dok se unutar listova skladište listovi B stabla. U slučaju tabela, u listovima se nalaze korisnički podaci, a svaki podataka u stranici sadrži veličinu reda, rowid (primarni ključ), podatke i overflow pointer. Kao što se može primetiti, SQLite koristi clustered indekse. Indeksi sadrže mapiranje između indeksiranih kolona i primarnog ključa (rowid).

Payload overflow stranice – kada je red previše veliki kako bi stao u jednu stranicu, on se „presipa“ na overflow stranice. Overflow stranice formiraju lančanu listu.

Pointer map stranice – postoje kako bi se pojednostavila implementacija auto_vacuum i incremental_vacuum funkcionalnosti. Kod drugih tipova stranica, postoji veza između child i parent stranica tako što parent stranica ima pokazivač na child stranicu. Pointer map stranice se koriste za čuvanje veza u suprotnom smeru, od child ka parent stranicama. [1]

SQLite skladišti šemu baze podataka u tabeli koja je prikazana na slici 1.

```
CREATE TABLE sqlite_schema(
  type text,
  name text,
  tbl_name text,
  rootpage integer,
  sql text
);
```

Slika 1- Tabela u kojoj se skladišti šema baze

2.1.2 Izolacija kod SQLite-a

Podrazumevani nivo izolacije kod SQLite baze je *serializable* i SQLite postiže ovaj nivo izolacija time što se upisi u bazu serijalizuju (osim u slučaju da je podešavanje PRAGMA read_uncommitted uključeno). U jednom trenutku može da se dešava samo jedan upis u bazu. Baza može da ima više otvorenih konekcija, ali svaka mora da sačeka svoj red. SQLite ovo postiže korišćenjem *lock-ova*, tako da aplikacija koja koristi SQLite ne mora da vodi računa o zaključavanju. Jedina situacija u kojoj jedna konekcija može da vidi nekomitovane promene druge konekcije je u slučaju kada je podešavanje read_uncommitted uključeno. [1]

SQLite može da radi u dva moda – *journal* mod i *wal* mod. Kod *journal* moda sve promene se upisuju direktno u osnovni fajl baze, ali se istovremeno i kreira pomoćni fajl u istom direktorijumu u kome se nalazi osnovni fajl baze u koji se upisuju koraci kako bi se promene nad glavnim fajlom poništile u slučaju *rollback-a* transakcije. U *wal* modu, promene se ne upisuju u glavni fajl baze već se upisuju u pomoćni *wal* fajl, a promene se iz *wal* fajla prenose u glavni fajl prilikom checkpoint operacije.

U *journal* modu, prilikom upisa promena u glavni fajl, on se zaključava čime se sprečavaju operacije čitanja i upisa koje potiču od drugih konekcija. Reader-i mogu da budu aktivni sve do trenutka kada je potrebno izvršiti flush ka glavnom fajlu, kada se writer-u daje ekskluzivni pristup fajlu. U *wal* modu dozvoljeno je da se upis i čitanje nad različitim konekcijama izvršava istovremeno. Obzirom da se sve promene upisuju u *wal* fajl, reader-i nastavljaju da vide stari sadržaj baze, koji je bio aktuelan pre operacije upisa koja se odvija. Kada se izvrši checkpoint operacija kojom se glavni fajl ažurira, sve aktivne read operacije nastavljaju da vide sadržaj koji je bio aktuelan u bazi pre write operacije. [1]

SQLite obezbeđuje izolaciju između različitih konekcija, ali ne postoji izolacija između operacija koje se odvijaju unutar iste konekcije. Ukoliko se nad istom konekcijom izvrši SELECT operacija nakon INSERT/UPDATE/DELETE operacija, sve promene prouzrokovane tim operacijama će biti vidljive. Ukoliko se SELECT operacija odvija paralelno sa operacijama koje modifikuju podatke, rezultat izvršenja SELECT operacije je nedefinisan. [1]

2.1.3 Funkcionalnosti SQLite-a

SQLite nudi veliki skup funkcionalnosti koje nudi bilo koja druga relacionalna baza podataka. To uključuje neograničen broj tabela, indeksa, trigeri i pogleda. Svaka tabela može da ima do 32000 kolona i neograničen broj redova (broj redova je ograničen samo veličinom fajla, koji može da bude maksimalno 281 TB). U pogledu indeksa, SQLite nudi podršku za multi-column indekse, korišćenje DESC i COLLATE funkcija, parcijalne indekse, indekse nad funkcijama, clustered indekse i covering indekse. Postoje CHECK, UNIQUE, NOT NULL i FOREIGN KEY ograničenja, ACID transakcije, ugnježdene transakcije, podupiti, sve vrste joina do 64 tabela, kao i operacije DISTINCT, ORDER BY, GROUP BY, HAVING, LIMIT, OFFSET, UNION, UNION ALL, INTERSECT, EXCEPT, funkcije agregacije. [1]

Pored „standardnih“ funkcionalnosti koje nude relacionalne baze podataka na osnovu starijih standarda, SQLite nudi podršku i za window funkcije, CTEs (common table expressions), uključujući i rekurzivne CTEs, UPSERT operacija, virtuelne tabele, IS operator, korisnički definisane funkcije, a kroz ekstenzije i full-text search, R-tree indekse i JSON kolone.

2.1.4 SQLite use-cases

Najčešći situacije u kojima je SQLite pogodan za korišćenje su:

- Embedded sistemi – obzirom da SQLite ne zahteva administraciju, izuzetno je pogodan za korišćenje kod mobilnih telefona, televizora, konzola, kamera, satova, automobila, aviona, dronova i IOT sistema, drugim rečima SQLite je idealno rešenje za „edge“.
- Application file format – veliki broj desktop (mobilnih) aplikacija koristi SQLite kako bi skladištio stanje aplikacije, što uključuje CAD alate, alate za finansijsku analizu, version control sistemi itd.
- Web-sajtovi – SQLite može bez puno problema da se koristi kao baza podataka kod sajta koji imaju oko sto hiljada zahteva dnevno
- Keširanje sadržaja iz enterprise relacionih baza podataka

Vredi pomenuti još jednom da je SQLite jedan od najkorišćenijih softvera danas, a procenjuje se da postoji oko 10^{12} SQLite aktivnih baza podataka.

Situacije u kojima SQLite nije idealni izbor su klijent-server aplikacije, zbog latencije fajl sistema, kao i bugova koji postoji kod implementacija zaključavanja fajla kod svih operativnih sistema, visoko konkurentne aplikacije, zbog serijalizacije write operacija, aplikacije koje rade sa ogromnom količinom podataka (> 281 TB). [1]

2.2 libSQL

LibSQL je fork SQLite projekta, koji održava Turso (kompanija). LibSQL je nastao kao posledica činjenice da je SQLite open-source, ali nije open-contribution. Ideja libSQL-a je potpuna backward kompatibilnost sa SQLite-om, pri čemu zajednica može aktivno da učestvuje u daljem razvoju. [2]

Funkcionalnosti koje libSQL uvodi, a koje trenutno nisu podržane kod SQLite-a su:

- `libsql_close_hook` – jedan od (manjih) problema koji postoji kod SQLite-a je da ekstenzije imaju problem sa uklanjanjem resursa koje su same kreirale, na primer keširani prepared statements, zbog redosleda kojim se one gase. LibSQL zbog toga uvodi `libsql_close_hook` koju može da se koristi kako bi korisnik sam mogao da definiše redosled prilikom gašenja. [3]
- Menjanje kolona – SQLite nudi vrlo ograničene mogućnosti po pitanju modifikacije već postojećih kolona i tabela. `ALTER TABLE` komanda kod SQLite-a je ograničena na preimenovanje tabele, preimenovanje kolone, dodavanje i uklanjanje nove kolone. LibSQL uvodi mogućnost modifikacije kolona, odnosno dodavanja type affinity-ja koloni, kao i dodavanje i uklanjanje ograničenja (strani ključ, check, default...). Jedno ograničenje ove dodatne funkcionalnosti libSQL-a je da će u slučaju dodavanja novog ograničenja to ograničenje biti primenjeno samo nove ili ažurirane redove, odnosno već postojeći redovi neće biti automatski validirani u odnosu na to ograničenje. [3]
- Definisanje funkcija korišćenjem WebAssembly-ja – SQLite nudi mogućnost kreiranja korisnički definisanih funkcija definisanjem callback funkcija korišćenjem drajvera za odgovarajući programski jezik, ili sam SQL. LibSQL uvodi novu mogućnost – definisanje funkcije korišćenjem WebAssembly-ja direktno u bazi. [3]

- Virtual WAL i proširenje WAL api-ja – kao što je naglašeno ranije, WAL mod kod SQLite-a omogućava da jedan writer može da radi paralelno sa većim brojem reader-a. WAL implementacija je „hardcoded“, odnosno SQLite ne nudi mogućnost custom WAL implementacije, bez promene source code-a i rekompajliranja baze. LibSQL modifikuje SQLite implementaciju tako da dozvoljava custom implementaciju WAL-a. [3] Ovakva promena neophodna je kako bi se omogućila upotreba SQLite-a u cloud-u, obzirom da se u takvom okruženju WAL upisuje na S3, Azure Blob Storage ili na neki drugi način.
- Native vector similarity search – SQLite može da se koristi kao vektorska baza podataka korišćenjem neke od open-source ekstenzija. Kod libSQL-a mogućnost skladištenja vektora i vektorska pretraga dolaze kao native funkcionalnost.

2.3 Turso

Turso (baza podataka) ima suprotan pristup u odnosu na libSQL – kompletan rewrite SQLite-a korišćenjem Rust programskog jezika. Ideja je ponovna implementacija baze koja bi bila slična SQLite-u sa modernijom arhitekturom koja podržava konkurentne upise, asinhrono IO operacije i slično. [2] Kao i libSQL projekat, Turso baza podataka je open-source i open-contribution, a pod okriljem je istoimene kompanije Turso i još uvek se nalazi u ranim fazama razvoja.

2.3.1 CDC

Jedna od funkcionalnosti koju Turso uvodi je native podrška za *CDC (change data capture)*. Mnoge baze podataka (PostgreSQL, MySQL, Oracle...) nude podršku za CDC u vidu strimovanja promena iz log fajlova (WAL, redo) koje se dešavaju, a eksterni alati kao što je *Debezium* mogu da konzumiraju takve strimove i da promene, u vidu događaja, objavljuju na drugo mesto, najčešće neki message broker kao što je Kafka.

Turso implementacija CDC-a je nešto drugačija. Sve izmene koje se dese nad tabelama pamte se u lokalnoj tabeli i mogu da se pretražuju kao bilo koja druga tabela. [2] Pored toga CDC se kod Turso-a podešava na nivou konekcije. Ograničenje koje postoji je da CDC ne može da se koristi u kombinaciji sa MVCC modom na nivou jedne konekcije.

CDC može da radi u nekoliko modova:

- id – pamti se samo primarni ključ promenjenih redova
- before – pamti se samo staro stanje promenjenih redova (kod update i delete operacija)
- after – pamti se samo novo stanje promenjenih redova (kod update i insert operacija)
- full – pamti se i staro i novo stanje promenjenih redova

Sve CDC promene se podrazumevano pamte u tabeli `turso_cdc`, a tabela ima strukturu prikazanu na slici 2.

CDC promene se generišu samo ako je transakcija uspešna, odnosno samo ako je commit-ovana. Ako se desi rollback transakcije ne generišu se CDC eventi. Svi redovi koji su promenjeni unutar iste transakcije imaju isti `change_txn_id`, a pored promenjenih redova generiše se i dodatni CDC event – COMMIT. Pored DML promena, CDC može da prati i DDL promene[2]

Kako je podešavanje CDC-a na nivou jedne konekcije, različite konekcije mogu da se koriste da se „hvataju“ promene nad različitim tabelama i u različitim modovima.

Column	Type	Description
<code>change_id</code>	INTEGER	Auto-incrementing unique identifier
<code>change_time</code>	INTEGER	Timestamp (Unix epoch)
<code>change_txn_id</code>	INTEGER	Transaction ID (groups rows into transactions)
<code>change_type</code>	INTEGER	<code>1</code> = INSERT, <code>0</code> = UPDATE, <code>-1</code> = DELETE, <code>2</code> = COMMIT
<code>table_name</code>	TEXT	Name of the table that was changed
<code>id</code>	varies	Primary key/rowid of the changed row
<code>before</code>	BLOB	Row data before the change (modes: <code>before</code> , <code>full</code>)
<code>after</code>	BLOB	Row data after the change (modes: <code>after</code> , <code>full</code>)
<code>updates</code>	BLOB	Per-column change details (mode: <code>full</code>)

Slika 2- Struktura tabele u koju se smeštaju CDC promene, preuzeto iz [2]

2.3.2 Konkurentni upis

Kako je Turso SQLite rewrite, pored svih noviteta koje uvodi, zadržava veliki broj podrazumevanih podešavanja i funkcionalnosti SQLite-a. Jedno od tih podešavanja je da u jednom trenutku samo jedna konekcija može da vrši upis u jednom trenutku. SQLite i u WAL modu ne dozvoljava da veći broj konekcija istovremeno vrši upis.

Iako je SQLite optimizovan i može lako da upisuje do 500k redova u sekundi sa odgovarajućom podelom u batch-eve, problem je u tome što se dodavanjem više thread-ova ne skalira količina upisa koja može da se prihvati. Performanse dodatno opadaju kako se količina CPU obrade u aplikaciji koja pristupa bazi radi upisa povećava. [4]

Turso uvodi MVCC (Multi-Version Concurrency Control) mod koji omogućava da veći broj konekcija vrši istovremeni upis. Ako transakcije pokušaju da modifikuju iste podatke, jedna od transakcija će dobiti grešku da je nastao konflikt i biće primorana da izvrši rollback. [2]

Već godinama, na eksperimentalnoj grani SQLite-a, postoji verzija koja nudi BEGIN CONCURRENT funkcionalnost koja u određenoj meri rešava problem konkurentnog upisa. Korišćenjem ovog pristupa, konekcije koje vrše upis odlažu pribavljanje lock-a do trenutka kada sa izdaje COMMIT naredba. Rešavanje konflikta je na nivou stranica, što znači da će jedna od konkurentnih transakcija biti primorana da izvrši rollback iako nije modifikovala iste podatke kao druga transakcija, već su nesrećnim okolnostima modifikovani redovi u istoj stranici. [4]

Turso u MVCC modu koristi sličan pristup. Za razliku od SQLite-a koji koristi *pessimistic locking* nad celom bazom kako bi se sprečili mogući konflikti, Turso koristi *optimistic locking* gde se tek kod commit-a proverava da li postoji konflikt na nivou redova. Turso ovo postiže tako što održava in-memory indeks koji prati verzije redova. [4]

Na slici 3 prikazane su tri transakcije koje se izvršavaju, a na slici 4 prikazan je izgled in-memory MVCC indeksa koji je kreiran. Red 1 ima dve verzije, verziju 1 koja je validna od transakcije T1 do transakcije T3, i verziju 2 koja je validna od verzije T3 na dalje. Red 2 ima jednu verziju koja je validna od transakcije T2 na dalje. MVCC indeks definiše i vidljivost redova unutar različitih transakcija. Prilikom pristupa nekom redu, ukoliko se on ne nalazi u MVCC indeksu, pristupa mu se sa diska, a u slučaju da već postoji, koristi se ta verzija, uz garanciju da je to poslednja vrednost reda. [4]

```
CREATE TABLE products (name, quantity);

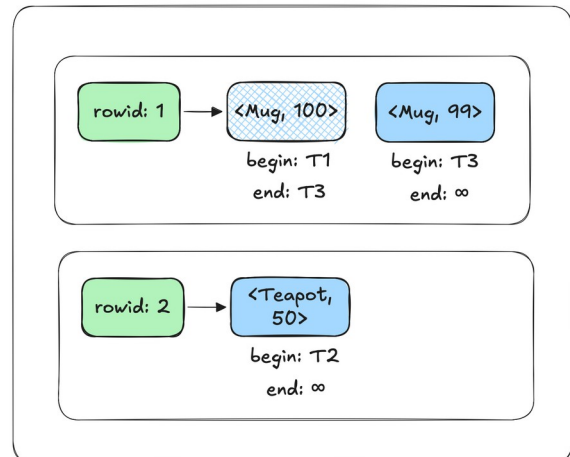
- Transaction T1
BEGIN CONCURRENT;
INSERT INTO products VALUES ('Mug', 100);
COMMIT;

- Transaction T2
BEGIN CONCURRENT;
INSERT INTO products VALUES ('Teapot', 500);
COMMIT;

- Transaction T3
BEGIN CONCURRENT;
UPDATE products SET quantity = quantity - 1 WHERE name = 'Mug';
COMMIT;
```

Slika 3- Primer transakcija

MVCC in-memory index



Slika 4- MVCC in-memory indeks pri izvršenju transakcija sa slike 4

Konkurentni upisi su i dalje eksperimentalna funkcionalnost kod Turso-a i ima svojih ograničenja. Skladištenje redova u indeksu je memorijski neefikasno, obzirom da se kao svaka verzija pamti ceo red. Ukoliko je promenjena samo jedna kolona u jednom redu, nova verzija reda u MVCC indeksu će svakako biti kompletan red. Pored toga, upravljanje verzijama redova nije wait-free operacija, obzirom da se verzije skladište u vektoru koji je zaštićen reader-writer lockom. Konačno, konkurentne transakcije ne koriste asinhroni io. [4]

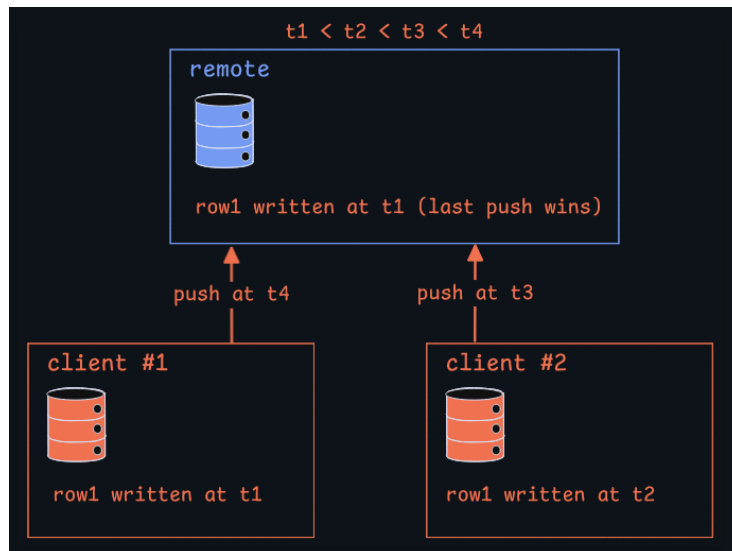
2.3.3 Sync

Skup tehnologija koje su poslednjih godina dobile na popularnosti su tehnologije koje omogućavaju razvoj offline-first aplikacija. Radi se o aplikacijama koje mogu da se izvršavaju bilo gde, bez obzira da li korisnik ima pristup internetu ili u datom trenutku nema. Offline-first aplikacija ne mora neophodno da bude korišćena od strane čoveka, podjednako verovatno može da se radi o edge uređaju koji se izvršava u nepovoljnim uslovima i privremeno je isključen sa mreže. Bez obzira na to, eventualno, kada aplikacija ponovo uspostavi vezu sa centralnim serverom ili drugim članovima mreže, može sinhronizovati svoje stanje sa ostalima. Najveći problem koji se rešava kod ovakvih aplikacija je rezolucija mogućih konflikata i u te svrhe razvijeni su različiti pristupi – last write wins, conflict free merging (CRDT i OT), izbegavanje konflikata itd.

Određeni skup baza podataka nudi mogućnost sinhronizacije replika. Jedna od najčešćih tehnika koja se koristi u ovim situacijama je korišćenje lokalne SQLite baze podataka i razvoj sync engine-a. Isti pristup koristi i Turso.

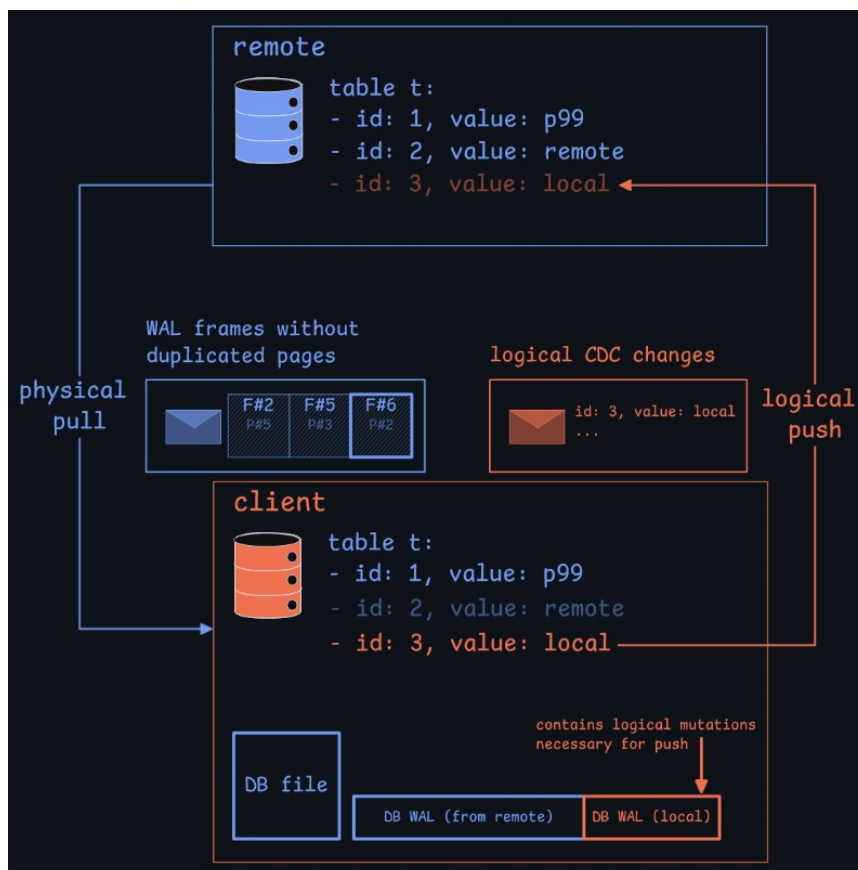
Turso koristi git terminologiju push i pull kako bi opisao proces sinhronizacije, a za rezoluciju konflikata koristi se *last-push-wins* strategija. To znači da ako je isti red modifikovan na različitim uređajima, verzija koja je poslednja sinhronizovana će biti aktuelna, bez obzira na vreme kada se ta promena desila. [5] Ilustracija last-push-wins strategije prikazana je na slici 5.

Last-push-wins je dovoljno dobra podrazumevana strategija za rezoluciju konflikata, ali u slučaju da aplikacija zahteva sofisticiraniju logiku, Turso nudi mogućnost definisanja custom logike koja će biti primenjena nad mutacijama koje se šalju na server, a podesiva je na nivou konekcije.



Slika 5- Ilustracija last-push-wins strategije za rezoluciju konflikata

Sinhronizacija kod Turso-a koristi hibridni pristup za sinhronizaciju – lokalne promene se push-uju na remote server kao logička promena, ali istovremeno promene sa remote servera mogu da se pull-uju kako bi lokalna replika eventualno postala identična kopija remote servera. Remote server se koristi kao *source of truth*, što znači da peer-to-peer sinhronizacija između uređaja trenutno nije podržana. [5] Detaljnija ilustracija pull i push procesa prikazana je na slici 6.



Slika 6- Ilustracija push i pull mehanizma kod Turso sync-a

3 Turso Cloud

Projekti libSQL i *rewrite* SQLite-a u Rust-u (Turso baza) su open-source, open-contribution projekti pod pokroviteljstvom kompanije Turso. Ovi projekti sami po sebi ne generišu novčane prihode i tu se u sliku pojavljuje Turso cloud. Radi se o *fully managed* SQLite-kompatibilnoj bazi podataka koja je zasnovana na libSQL-u. Trenutno, Turso Cloud koristi libSQL, ali je plan da sledeća generacija Turso Cloud-a u potpunosti koristi Turso bazu podataka. Neke od funkcionalnosti koje Turso Cloud nudi su native podrška za vektorsku pretragu, replikacija i sync, grananje (*branching*), analitika, visoko granularni tokeni za pristup, enkripcija podataka ključevima kojima samo vlasnik ima pristup itd. [6]

3.1 Cloud specifikacija

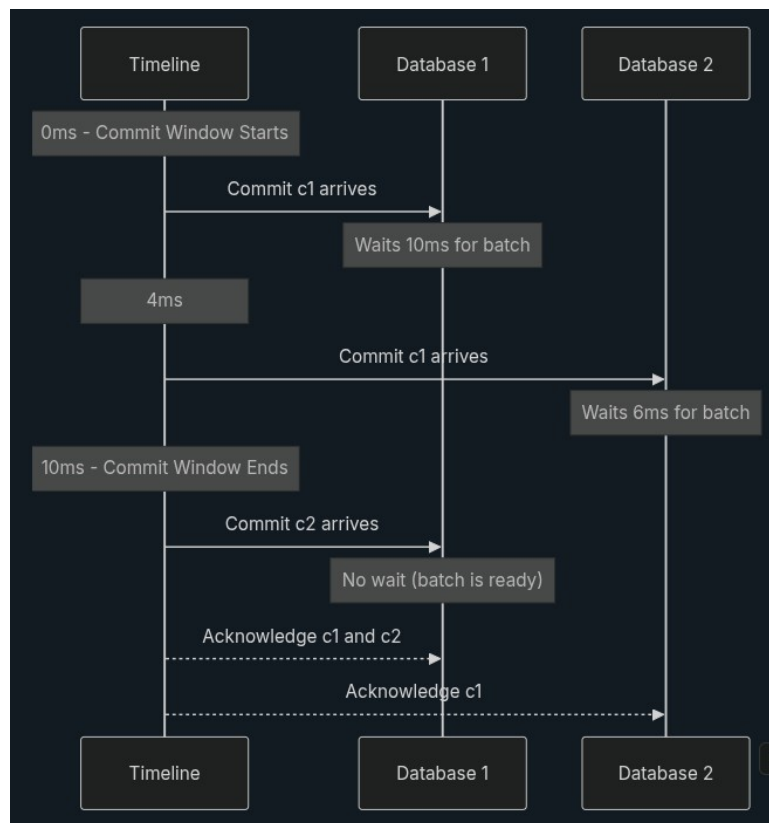
Turso Cloud u pozadini koristi AWS te je većina garancija koje Turso Cloud daje usko povezana sa garancijama koje AWS nudi.

Turso Cloud garantuje 99.999999999% *durability* podataka (11 nines). Kao što je pomenuto, Turso ovo postiže tako što koristi kombinaciju AWS S3-Express One Zone i AWS S3 tehnologija. Korisniku baze se commit potvrđuje tek nakon što je podatak uspešno upisan bilo u S3 ili S3-express. Turso Cloud ovakav pristup zove diskless arhitekturom – podaci se trajno skladište samo na AWS-u, dok se lokalni čvorovi koriste za compute i keširanje, te iz tog razloga Turso može da pruži isti nivo *durability*-ja kao i AWS. [6]

Svaka SQLite kompatibilna baza podataka ima dve glavne komponente – WAL koji sadrži poslednje commit-ove i glavni fajl. Periodično se podaci sa WAL-a upisuju u glavni fajl baze. Novi commit-ovi u bazi se skladište u S3-express-u pre nego što se izda potvrda da je podatak uspešno zapisan. Periodično, podaci se prebacuju u glavni fajl koji se skladišti na S3. [6]

AWS S3-Express One Zone i AWS S3 su object-store servisi koji su optimizovani za različite use caseve. Ključna razlika je što standardni S3 replicira podatke kroz više zona dostupnosti (*availability zones*) u AWS regionu. Projektovan je tako da podaci mogu da prežive pad cele zone. Sa druge strane S3-Express skladišti podatke u jednoj zoni dostupnosti. Na taj način se eliminiše potreba za međuzonskom komunikacijom. Sa stanovišta performansi, baš zbog odsustva međuzonske komunikacije S3-Express na jednom čvoru ima brzinu koja je bliska regularnom fajl sistemu, a distribuirana priroda servisa obezbeđuje obrađivanje stotine hiljada do nekoliko miliona zahteva u sekundi. To uključuje brže kreiranje, pribavljanje i brisanje objekata i nižu latenciju zahteva.

U zavisnosti od cloud tier-a koji korisnik plaća, latencija potvrde commit-a može da se kreće između 10ms i 100ms. AWS S3 nije popularan izbor za OLTP workload zbog cene upisa u object store. Turso Cloud koristi multi-tenant arhitekturu pa cenu upisa u S3 amortizuje grupisanjem (*batching*) commitova, što predstavlja značajan udeo u latentnosti potvrde commit-a. Na slici 7 prikazana je ilustracija latencije koja nastaje usled grupisanja. U trenutku 0, kada je otvoren commit prozor, stiže commit zahtev. Obzirom da commit prozor traje 10ms, taj commit zahtev će morati da sačeka svih 10ms pre nego što bude potvrđen. Nakon toga, 4ms nakon otvaranja commit prozora stiže novi commit zahtev, i on će morati da sačeka 6ms pre potvrđivanja. Konačno, do baze 1 stiže commit neposredno nakon završetka commit prozora, te kako je batch već spreman taj commit neće imati dodatnu latenciju usred grupisanja. Ova latencija se dodaje na latenciju koja postoji usled upisa na AWS S3 i na taj način se dobija ukupna latencija zahteva. Latencija čekanja usled grupisanja varira u zavisnosti od nivoa cloud-a koji korisnik poseduje. [6]



Slika 7- Ilustracija latencije usled grupisanja commit-ova

Turso Cloud nudi nekoliko nivoa – free, developer, scaler i pro. Na slici 8 prikazane su cene ovih nivoa kao i pregled osnovnih ograničenja koje taj nivo nudi u trenutku pisanja rada.

	Free	Developer	Scaler	Pro
	\$0	\$4.99 /month	\$24.92 /month	\$416.58 /month
	Sign up	Get Developer	Get Scaler	Get Pro
Databases	100	∞ Unlimited	∞ Unlimited	∞ Unlimited
Storage	5GB	9GB + \$0.75 / GB	24GB + \$0.50 / GB	50GB + \$0.45 / GB
Monthly Rows Read	500 Million	2.5 Billion + \$1 / Billion	100 Billion + \$0.80 / Billion	250 Billion + \$0.75 / Billion
Monthly Rows Written	10 Million	25 Million + \$1 / Million	100 Million + \$0.80 / Million	250 Million + \$0.75 / Million
Monthly Syncs	3GB	10GB + \$0.35 / GB	24GB + \$0.25 / GB	100GB + \$0.15 / GB
Point-In-Time Restore	✓ 1 day	✓ 10 days	✓ 30 days	✓ 90 days

Slika 8- Turso Cloud pricing

Kao i praktično svaki cloud provajder, Turso nudi mogućnost ograničavanja količine iskorišćenih resursa kako se cena ne bi premašila. Podrazumevano, ovo podešavanje je isključeno, što znači da u slučaju prekoračenja ograničenja koja nudi određeni nivo, baza podataka ostaje dostupna ali mesečna cena nastavlja da raste bez ograničenja. U slučaju da je ovo podešavanje uključeno, kada se dostigne limit za taj mesec baza podataka će biti blokirana. [6]

Rows read i rows written parametri sa slike 8 su proxy za compute. Svaki red kom se pristupi prilikom određivanja rezultata se računa ka ovom broju, a ne samo redovi koji su vraćeni aplikaciji. Kod čitanja ovo znači da pretraga po neindeksiranoj koloni mora da skenira čitavu tabelu, tako da se svaki red iz tabele računa ka ovom ograničenju. Pretraga koja koristi indeks može da rezultuje time da se pročita samo 1 red – što još jednom pokazuje značaj korišćenja indeksa. Za operacije koje modifikuju podatke – INSERT, UPDATE i DELETE, nad tabelama sa indeksima, često je potrebno ažurirati više redova jer svaki indeks mora da se ažurira. Dodavanje indeksa povećava cenu DML operacija. Zbog toga je važno da se indeksi pažljivo biraju. [6]

Na reads read i reads written parametre utiče i upotreba funkcija agregacije – count, avg, min, max, sum. Preporuka Turso-a je da se agregirane vrednosti skladište u pomoćnoj tabeli koja se povremeno ažurira. Pored toga na parameter još utiču i ugnježdjeni upiti i join između tabela. U obračun ulaze i necommitovane transakcije i izvršavanje alter table operacija. Ažuriranje sistemskih tabela (dbstat i tabele sa prefiksom sqlite_) ne utiče na parametre.

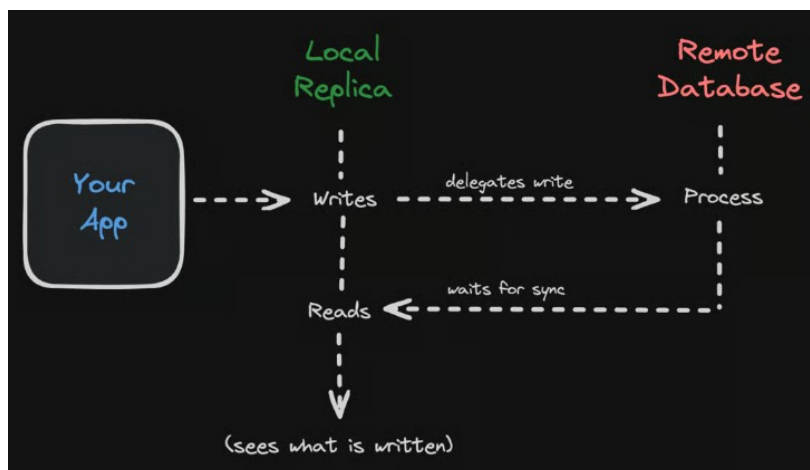
Pored compute-a koji je opisan u prethodnom pasusu i skladišta čije računanje je jasno, dodatno na cenu utiče i embedded sync (libSQL način za sync, koji se nešto razlikuje od Turso načina koji je opisan u prethodnom poglavlju). Embedded syncs mere količinu podataka koja se razmeni između lokalne replike i cloud baze. Point-in-time restores takođe donosi dodatne troškove, tako da svaki point-in-time restore zauzima jednu od slobodnih baza. Slično je i sa grananjem (*branching*), gde svaka grana zauzima jednu slobodnu bazu. [6]

3.2 Embedded replicas

U poglavlju 2.3.3 bilo je reči o sync funkcionalnosti koju nudi Turso baza podataka. Obzirom da u trenutku pisanja rada Turso Cloud koristi libSQL, u ovoj sekciji će biti opisano kako sync funkcionalnost radi u cloudu. Vredi napomenuti još jednom da je cilj Turso-a da u jednom trenutku svoj cloud prebace na Turso bazu podataka, čime će sync kakav je opisan u poglavlju 2.3.3 postati aktuelan i u cloud-u. U ovom trenutku moguće je koristiti i Turso database sync, ali za produkciona okruženja i dalje se preporučuje embedded replicas.

Ideja iza embedded replika je da se lokalno skladišti read replika, čime se omogućava da vreme izvršenja operacija čitanja bude reda mikrosekundi, a da se upisi šalju u cloud nakon čega se reflektuju u lokalnoj replici. Ilustracija rada embedded replika prikazana je na slici 9.

Sinhronizacija između lokalne replike i remote replike može da se vrši na dva načina. Prvi način je automatska periodična sinhronizacija. Alternativa nosi naziv „read your writes“. Ovaj način garantuje istoimeni nivo konzistentnosti kod distribuiranih sistema, odnosno da nakon upisa nekog podatka, naredno čitanje će vratiti prethodno upisanu vrednost. [6]



Slika 9- Ilustracija rada embedded replika

3.3 Branching

Grananje baza podataka je koncept sličan grananju koda kod sistema za upravljanje verzijama, gde se svakoj funkcionalnosti ili developeru daje posebna, izolovana kopija baze kako bi mogao da radi izmene, migracije i testiranja bez uticaja na druge ili na zajedničko okruženje. Umesto oslanjanja na jednu staging bazu koja se vremenom razilazi od produkcije, timovi sve češće prave privremene, grane-baze koje se automatski kreiraju za potrebe pojedinačnih taskova ili pull request-ova, zatim se u njima testiraju promene sa realističnim podacima i na kraju se brišu kada se posao završi. Na taj način se smanjuju konflikti u šemi i podacima, povećava sigurnost pri izmenama, omogućava paralelan rad više ljudi i lakše reprodukuju bagovi, dok produkcija ostaje „čista“ od nedovršenih promena. Grananje baza postalo je jako popularno sa „alternativnim“ bazama podataka u cloudu kao što su Neon, Supabase, Xata i Turso. [7]

Neon implementacija grananja je „najčistija“. Grana je kod Neon baze pokazivač sa upisima koji se dešavaju na toj grani. Nepromenjeni podaci u odnosu na osnovnu granu (odnosno bazu) su deljivi sa pomoćnim granama. Supabase grananju pristupa nešto drugačije, gde je svaka grana kompletan klon osnovne grane. Xata grananju pristupa sa stanovišta migracija šeme i nije preterano korisna kod CRUD workload-a. Konačno, kod Turso-a grana je zapravo embedded replika, lokalna kopija libSQL baze podataka. [7]

3.4 Point-in-Time recovery

Zbog načina na koji Turso Cloud skladišti podatke (korišćenjem AWS S3-Express ili AWS S3) ne postoji razlika između upisa u bazu podataka i kreiranja backup-a baze podataka. Backup se dešava uvek, kontinuirano i moguće je bazu podataka vratiti u bilo koje prethodno stanje u poslednjih 90 dana. [8] Korišćenje point-in-time recovery mehanizma kreira novu instancu baze podataka, čime se menja, između ostalog i connection string za pristup bazi, kao i tokeni za pristup. [6]

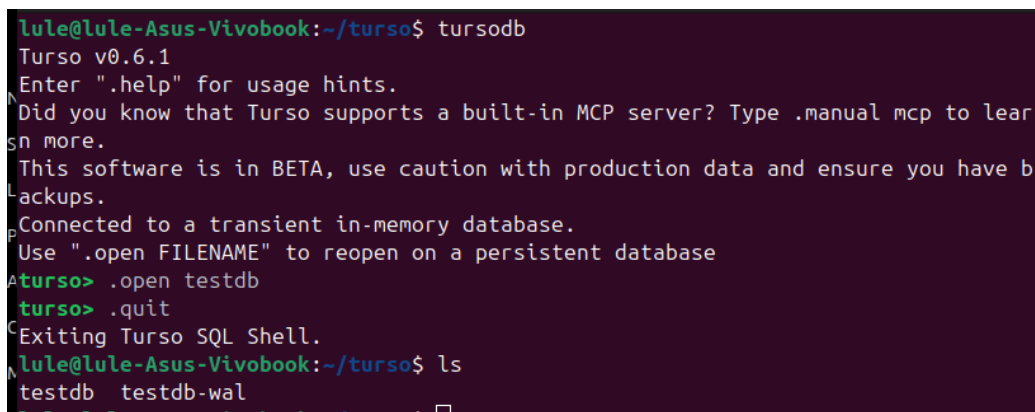
Turso glavni database fajl ne čuva kao jedan fajl, već je podeljen u segmente veličine 128kB. Point-in-time recovery zapravo pronalazi segmente fajla koji su bili aktuelni u datom trenutku i po potrebi ponavlja zapise koji se nalaze u WAL fajlu. Kako Turso garantuje point-in-time recovery do 90 dana u prošlosti, to implicitno znači da se WAL fajlovi skladište minimalno 90 dana. Zbog toga se segmenti i WAL zajedno koriste za rekonstrukciju stanja baze, pri čemu segmenti daju „osnovu“ istorije, a WAL omogućava precizno vraćanje na tačan trenutak između dve verzije.

4 Praktični prikaz

4.1 Turso database

U prvom delu praktičnog priakza biće prikazane funkcionalnosti Turso baze podataka, SQLite rewrite-a u Rust-u. Turso, kao i sam SQLite, može da se izvršava na praktično bilo kom operativnom sistemu, pa čak i u sistemima koji se izvršavaju bez operativnog sistema. Tokom ovog praktičnog prikaza korišćen je Ubuntu Linux operativni sistem.

Bazi podataka je moguće pristupiti iz konzole pokretanjem programa *tursodb*. Prilikom pokretanja korisnik se nalazi u „in-memory“ modu, gde će svi podaci biti izgubljeni prilikom gašenja konzole. Za kreiranje perzistentne baze podataka, u konzoli se izvršava komanda *.open <filename>*, nakon čega se kreira glavni fajl baze na disku. Na slici 10 prikazano je izvršenje ovih komandi, kao i sadržaj direktorijuma iz kog je pokrenuta *tursodb* operacija.



```
lule@lule-Asus-Vivobook:~/turso$ tursodb
Turso v0.6.1
Enter ".help" for usage hints.
Did you know that Turso supports a built-in MCP server? Type .manual mcp to learn more.
This software is in BETA, use caution with production data and ensure you have backups.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database
A turso> .open testdb
turso> .quit
C Exiting Turso SQL Shell.
lule@lule-Asus-Vivobook:~/turso$ ls
testdb  testdb-wal
```

Slika 10- Pokretanje tursodb baze i kreiranje glavnog fajla baze

Korišćenjem konzole korisnik može da izvršava sve standardne SQL operacije i da koristi sve napredne funkcionalnosti koje Turso nudi, međutim preporučeni način za rad sa Turso bazom podataka je korišćenjem driver-a u nekom programskom jeziku. Programski jezici koji su u trenutku pisanja rada podržani su JavaScript, Python, Go, Rust, Dart i Java, a u okviru ovog rada biće korišćen Python. Biblioteka za rad sa SQLite-om je ugrađena u sam jezik, u okviru biblioteke *sqlite3*, a biblioteka za rad sa Turso bazom se instalira kao posebna biblioteka *pyturso* i implementira standardan Python api za rad sa bazom podataka. Primer rada sa ovim API-jem korišćenjem *pyturso* biblioteke prikazan je na slici 11.

```
import turso

con = turso.connect("/home/lule/turso/luca.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS accounts (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    type TEXT NOT NULL,
    currency TEXT NOT NULL,
    created_at TEXT NOT NULL
);
""")
con.commit()

cur.executemany("""
INSERT INTO accounts (id, name, type, currency, created_at)
VALUES (?, ?, ?, ?, ?)
""", [
    (1, "Main Checking", "checking", "USD", "2026-06-29T09:00:00Z"),
    (2, "Vacation Savings", "savings", "USD", "2026-06-29T09:05:00Z"),
])

con.commit()
```

Slika 11- Primer korišćenja *pyturso* biblioteke

4.1.1 CDC

Za demonstraciju CDC mehanizma koji Turso nudi, pored tabele prikazane na slici 11 koristiće se i tabela sa slike 12. Primer programa koji se izvršava i popunjava ovu tabelu prikazan je na slici 13. CDC će biti prikazan u *full* modu, koji pamti i vrednosti redova pre i nakon izmene.

```
turso> CREATE TABLE transactions (
  id INTEGER PRIMARY KEY,
  account_id INTEGER NOT NULL,
  amount INTEGER NOT NULL, -- cents
  description TEXT,
  transaction_date TEXT NOT NULL,
  created_at TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'posted', -- pending, posted, failed, reversed
  FOREIGN KEY (account_id) REFERENCES accounts(id)
);
```

Slika 12- Tabela koja pamti transakcije korisnika

```
import turso
from datetime import datetime, timezone

con = turso.connect("/home/lule/turso/luka.db")
cur = con.cursor()

cur.execute("PRAGMA capture_data_changes_conn('full');")
cur.execute("PRAGMA foreign_keys = ON;")

now = datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")
tx_id = 1001

cur.execute(
    """
    INSERT INTO transactions (id, account_id, amount, description, transaction_date, created_at, status)
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """,
    (tx_id, 1, -2599, "Coffee and lunch", "2026-06-29", now, "pending"),
)

cur.execute(
    """
    UPDATE transactions
    SET status = ?
    WHERE id = ?
    """,
    ("posted", tx_id),
)

con.commit()
```

Slika 13- Primer operacija nad tabelom transactions

Vrlo je važno naglasiti da se CDC kod Turso-a podešava na nivou konekcije, za razliku od većine drugih baza podataka gde se CDC uključuje na nivou tabele/šeme/baze. Sve CDC promene Turso skladišti u tabeli *turso_cdc*, a šema ove tabele prikazana je na slici 2. Nakon izvršenja programa sa slike 13 i pretraživanja ove tabele dobijaju se rezultati prikazani na slici 14.

```
turso> select * from turso_cdc;
```

change_id	change_time	change_txn_id	change_type	table_name	id	before	after	updates
1	17827...	1	1	trans...	1001		-...	
2	17827...	1	0	trans...	1001	-...	-...	
3	17827...	1	2					

Slika 14- Sadržaj turso_cdc tabele nakon izvršavanja programa sa slike 13

Kolona *change_id* je jedinstveni identifikator promene, *change_time* vreme kada se ta promena desila, *change_txn_id* je id transakcije u kojoj je nastala promena, *change_type* je tip promene (insert, update, delete ili commit), *table_name* naziv tabele nad kojom se promena desila, id je primarni ključ u tabeli, a kolone before i after vrednost reda pre i posle operacije. Kolona updates prikazuje promene na nivou kolone.

Turso nudi veliki broj pomoćnih funkcija za detaljniju inspekciju CDC tabel, obzirom da se before i after kolone pamte kao kompresovani binarni zapisi. Primer upita koji koristi pomoćne funkcije kako bi se kolone before i after prikazale u json zapisu prikazan je na slici 15, a rezultat izvršenja ovog upita na slici 16.

```
SELECT
  change_id,
  change_txn_id,
  CASE change_type
    WHEN 1 THEN 'INSERT'
    WHEN 0 THEN 'UPDATE'
    WHEN -1 THEN 'DELETE'
    WHEN 2 THEN 'COMMIT'
  END AS operation,
  table_name,
  id,
  bin_record_json_object(
    table_columns_json_array('transactions'),
    before
  ) AS before_state,
  bin_record_json_object(
    table_columns_json_array('transactions'),
    after
  ) AS after_state
FROM turso_cdc
WHERE table_name = 'transactions'
ORDER BY change_id;
```

Slika 15- Upit kako bi se zapisi iz cdc tabele prikazali u formatu koji je razumniji za ljude

change_id	change_txn_id	operation	table_name	id	before_state	after_state
1	1	INSERT	transactions	1001		{"id":1001,"account_id":1,"amount":-2599,"description":"Cof.
2	1	UPDATE	transactions	1001	{"id":1001,"account_id":1,"amount":-2599,"description":"Coff.	{"id":1001,"account_id":1,"amount":-2599,"description":"Cof.

Slika 16- Rezultat izvršenja upita sa slike 15

Ova funkcionalnost je u suštini bolji način za implementaciju trigger-based *CDC-a*. Nedostatak ovakvog pristupa je problem sa streamovanjem ovih podataka. Debezium još uven nema podršku za CDC, a sam Turso/SQLite nema *Advanced Queueing* mehanizam kao što ima Oracle. Zbog toga ukoliko je potrebno streamovati ovakve podatke neophodno je implementirati polling mehanizam koji bi pribavljao podatke iz CDC tabele na osnovu vremena kada su se desili. Ni jedna kolona u tabeli *turso_cdc* nije indeksirana pa bi u toj situaciji indeks bio neophodan bar nad kolonom *change_time*. Kada Turso postane podrazumevana baza u Turso cloud-u kreiranje ovakvih indeksa unosi dodatne troškove što komplikuje implementaciju enterprise CDC sistema.

Pored promena nad podacima CDC kod Turso-a ima mogućnost da prati i promene nad šemama tabela. Neophodno je da CDC na nivou konekcije bude u *full* modu, a sve promene se pamte u tabeli *turso_cdc*, kao promene nad tabelom *sqlite_schema*. Za demonstraciju ove funkcionalnosti izvršeni su upiti sa slike 17, a izvršenja upita sa slike 15, gde je *table_name* promenjen na „*sqlite_schema*“ prikazan je na slici 18, a na slici 19 prikazan je before i after ALTER TABLE upita.

```
CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT);
ALTER TABLE products ADD COLUMN price INTEGER NOT NULL DEFAULT 0;
DROP TABLE products;
```

Slika 17- Primer DDL naredbi

change_id	change_txn_id	operation	table_name	id	before_state	after_state
4	4	INSERT	sqlite_schema	7		{"type": "table", "name": "products", "tbl_name": "products", "roo...
6	6	UPDATE	sqlite_schema	7	{"type": "table", "name": "products", "tbl_name": "products", "roo...	{"type": "table", "name": "products", "tbl_name": "products", "roo...
8	8	DELETE	sqlite_schema	7	{"type": "table", "name": "products", "tbl_name": "products", "roo...	

Slika 18- Primer izvršenja upita sa slike 17

```
{
  "type": "table",
  "name": "products",
  "tbl_name": "products",
  "rootpage": 8,
  "sql": "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT)"
}

{
  "type": "table",
  "name": "products",
  "tbl_name": "products",
  "rootpage": 8,
  "sql": "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price INTEGER NOT NULL DEFAULT 0)"
}
```

Slika 19- Primer *before* i *after* nakon izvršenja ALTER table operacije

Kako je podešavanje CDC-a na nivou konekcije, prilikom podešavanja CDC nivoa moguće je specificirati specifičnu tabelu koja će se pratiti, tako što se nakon specificiranja CDC nivoa sa zarezom odvoje tabele. Primer je prikazan na slici 20.

```
turso> PRAGMA capture_data_changes_conn('full,accounts');
turso> PRAGMA capture_data_changes_conn('full,accounts,transactions');
```

Slika 20- Primer podešavanja tabela nad kojima će CDC biti omogućen

4.1.2 Konkurentni upisi

Kao što je detaljnije opisano u istoimenom poglavlju 2.3.2, Turso uvodi mogućnost konkurentnih upisa sa detekcijom konflikata na nivou pojedinačnih redova. Kako bi konkurentni upisi bili omogućeni, potrebno je na nivou konekcije postaviti MVCC mod, a zatim prilikom izvršenja upisa koristiti BEGIN CONCURRENT komandu.

Na slici 21 prikazan je kod koji se izvršava konkurentno od strane većeg broja konekcija. Svaka konekcija otvara zasebnu transakciju i pokušava da upiše vrednost u tabelu *hits*. Cilj primera nije da se prikaže različit krajnji sadržaj tabele, već razlika u načinu izvršavanja konkurentnih upisa sa i bez MVCC režima.

U slučaju bez MVCC režima koristi se standardni mehanizam zaključavanja, gde samo jedna konekcija može da izvršava upis u datom trenutku. Zbog toga ostale konekcije nailaze na grešku *database is locked*, pa samo jedna transakcija uspešno završava izvršavanje. Sa druge strane, kada je uključen MVCC režim i kada se koristi BEGIN CONCURRENT, više konekcija može istovremeno da drži aktivne transakcije za upis. Pošto se u ovom primeru upisuju nezavisni redovi, ne dolazi do konflikta nad istim podacima i sve transakcije se uspešno završavaju. U primeru je iskorišćena BEGIN IMMEDIATE komanda kojom transakcija odmah pribavlja write lock. Obzirom da su

pojedinačne insert operacije jako brze, ova komanda je pogodna za forsiranje zaključavanja i demonstraciju pogodnosti koje MVCC mehanizam uvodi.

Za pokretanje većeg broja konkurentnih konekcija korišćena je asyncio biblioteka, pri čemu se svaka konekcija izvršava u posebnoj niti. Deo koda prikazan je na slici 22, dok je rezultat izvršavanja prikazan na slici 24. Dobijeni rezultat pokazuje da se bez MVCC režima uspešno izvršava samo jedna transakcija, dok se u MVCC režimu uspešno izvršavaju sve konkurentne transakcije. Razlika je, dakle, u tome što MVCC omogućava stvarno preklapanje konkurentnih upisa, dok se u standardnom režimu upisi međusobno blokiraju.

```
def run_worker(db_path: str, mode: str, worker_id: int, start_at: float) -> tuple[int, float, str]:
    conn = turso.connect(db_path)

    try:
        if mode == "mvcc":
            conn.execute("PRAGMA journal_mode = 'mvcc'").fetchone()

        t0 = time.perf_counter()

        try:
            if mode == "mvcc":
                conn.execute("BEGIN CONCURRENT")
            else:
                conn.execute("BEGIN IMMEDIATE")

            conn.execute("INSERT INTO hits VALUES (?)", (worker_id,))
            time.sleep(SLEEP_INSIDE_TX)
            conn.execute("COMMIT")

            elapsed = time.perf_counter() - t0
            return worker_id, elapsed, "committed"

        except Exception as e:
            try:
                conn.execute("ROLLBACK")
            except Exception:
                pass

            elapsed = time.perf_counter() - t0
            return worker_id, elapsed, f"failed: {type(e).__name__}: {e}"

    finally:
        conn.close()
```

Slika 21- Kod koji se izvršava od strane većeg broja konkurentnih konekcija

```
if mode == "mvcc":
    setup.execute("PRAGMA journal_mode = 'mvcc'").fetchone()

setup.execute("CREATE TABLE hits (val INTEGER)")
setup.commit()
setup.close()

start_at = time.perf_counter() + 0.5
t0 = time.perf_counter()

tasks = [
    asyncio.to_thread(run_worker, db_path, mode, i, start_at)
    for i in range(N_WORKERS)
]

results = await asyncio.gather(*tasks)
total_elapsed = time.perf_counter() - t0

final = turso.connect(db_path)

if mode == "mvcc":
    final.execute("PRAGMA journal_mode = 'mvcc'").fetchone()

count = final.execute("SELECT COUNT(*) FROM hits").fetchone()[0]
final.close()
```

Slika 22- Deo koda koji pokreće veći broj threadova

```

=====
mode: normal
rows inserted: 1
wall time: 0.260s
=====
worker=0 elapsed=0.257s committed
worker=1 elapsed=0.000s failed: OperationalError: database is locked
worker=2 elapsed=0.000s failed: OperationalError: database is locked
worker=3 elapsed=0.000s failed: OperationalError: database is locked
worker=4 elapsed=0.000s failed: OperationalError: database is locked
worker=5 elapsed=0.000s failed: OperationalError: database is locked
worker=6 elapsed=0.000s failed: OperationalError: database is locked
worker=7 elapsed=0.000s failed: OperationalError: database is locked

=====
mode: mvcc
rows inserted: 8
wall time: 0.276s
=====
worker=0 elapsed=0.257s committed
worker=1 elapsed=0.259s committed
worker=2 elapsed=0.255s committed
worker=3 elapsed=0.263s committed
worker=4 elapsed=0.256s committed
worker=5 elapsed=0.256s committed
worker=6 elapsed=0.261s committed
worker=7 elapsed=0.257s committed

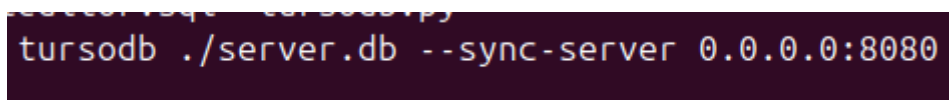
```

Slika 23- Rezultat izvršenja koda sa slike 21 i 22

4.1.3 Sync

Sync je jedna od najisčekivanijih funkcionalnosti koje je Turso baza podataka implementirala. Trenutno je podržan i u cloud-u, sa napomenom da je u takvoj postavci još uvek eksperimentalne prirode i ne preporučuje se korišćenje u produkciji. Umesto toga, tokom razvoja i radi testiranja sync funkcionalnosti Turso obezbeđuje lokalni sync server. Ovaj server implementira isti protokol kao i cloud server tako da aplikacija radi bez promena u oba okruženja.

Pokretanje sync servera vrši se komandom koja je prikazana na slici 24, a primer povezivanja klijenta na sync server prikazan je na slici 25.



```
tursodb ./server.db --sync-server 0.0.0.0:8080
```

Slika 24- Pokretanje lokalnog sync servera

```

import turso.sync

conn = turso.sync.connect(
    path="./local-replica.db",
    remote_url="http://localhost:8080",
)

```

Slika 25- Povezivanje na lokalni sync server

Nakon povezivanja na (lokalni) sync server, mogu se izvršavati push i pull operacije. Na slikama 26 i 27 prikazan je osnovni način na koji se koristi ova funkcionalnost. Dva klijenta su povezana na sync server, jedan klijent je napravio izmene nakon čega je pozvao push, a drugi poziva pull nakon čega vidi promene prvog klijenta.

```
import turso.sync

client_a = turso.sync.connect(
    path="./client-a.db",
    remote_url="http://localhost:8080",
)

client_a.execute("CREATE TABLE IF NOT EXISTS notes (id TEXT PRIMARY KEY, body TEXT)")
client_a.commit()
client_a.execute("INSERT INTO notes VALUES ('n1', 'hello from client A')")
client_a.commit()

client_a.push()
```

Slika 26- Primer korišćenja push operacije, prvi klijent

```
import turso.sync

client_b = turso.sync.connect(
    path="./client-b.db",
    remote_url="http://localhost:8080",
)

changed = client_b.pull()

rows = client_b.execute("SELECT * FROM notes").fetchall()
print(rows)
# [('n1', 'hello from client A')]
```

Slika 27- Primer korišćenja pull operacije, drugi klijent

Kao što je navedeno u poglavlju 2.3.3 za rezoluciju konflikata Turso koristi *last push wins* strategiju. To znači da kada dva klijena modifikuju isti podatak i urade push, poslednji push određuje stanje podatka na remote serveru. Na slici 28 je prikazan kod koji sada izvršava prvi klijent, dok drugi klijent izvršava kod analogno prvom klijentu.

```
client_a.execute("UPDATE NOTES set body = 'UPDATE FROM CLIENT A' WHERE id = 'n1'")
client_a.commit()
```

```
wait_until(23, 00)
client_a.push()
```

Slika 28- Kod koji paralelno izvršavaju dva klijenta

U zavisnosti od redosleda kojim se izvrši push operacija nova vrednost podatka može da bude bude „UPDATE FROM CLIENT A“ ili „UPDATE FROM CLIENT B“. U konkretnom primeru desilo se da drugi klijent ima poslednji push, tako da ako sada prvi klijent uradi pull, a nakon toga pribavi podatak biće prikazana izmena drugog klijenta, što je prikazano na slici 29.

```
11 client_a.pull()
12 rows = client_a.execute("SELECT * FROM notes").fetchall()
13 print(rows)
14
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
(venv)	lule@lule-Asus-Vivobook:~/Desktop/DBMS/P3/impl\$		python sync.py	
(venv)	lule@lule-Asus-Vivobook:~/Desktop/DBMS/P3/impl\$		python sync.py	
			[('n1', 'UPDATE FROM CLIENT B')]	
(venv)	lule@lule-Asus-Vivobook:~/Desktop/DBMS/P3/impl\$		□	

Slika 29- Pull operacija prvog klijenta

Kada postoje izmene koje još uvek nisu push-ovane, a izvrši se pull operacija, baza se rollback-uje u poslednje sync stanje, primenjuju se remote izmene, a nakon toga se replay-uju lokalne izmene.

Poslednja osobina sync-a koja će biti prikazana je mogućnost kreiranja custom callback funkcije koja će se izvršiti prilikom slanja update-a na remote server. Ova opcija još uvek nije dostupna u Python biblioteci, pa je iz tog razloga primer prikazan u TypeScript-u.



```
const db = await connect({
  path: "./client-a.db",
  url: "http://localhost:8080",

  transform: (mutation: any) => {
    if (mutation.table !== "counter") {
      return { operation: "keep" };
    }

    if (!mutation.before || !mutation.after) {
      return { operation: "keep" };
    }

    const beforeValue = mutation.before.value;
    const afterValue = mutation.after.value;

    if (
      typeof beforeValue !== "number" ||
      typeof afterValue !== "number" ||
      beforeValue === afterValue
    ) {
      return { operation: "keep" };
    }

    const delta = afterValue - beforeValue;

    return {
      operation: "rewrite",
      stmt: {
        sql: `
          UPDATE counter
          SET value = value + ?
          WHERE key = ?
        `,
        values: [delta, mutation.after.key],
      },
    };
  },
});

await db.exec(`
  CREATE TABLE IF NOT EXISTS counter (
    key TEXT PRIMARY KEY,
    value INTEGER NOT NULL
  )
`);

await db.exec(`
  INSERT OR IGNORE INTO counter(key, value)
  VALUES ('likes', 0)
`);

await db.pull();

await db.exec(`
  UPDATE counter
  SET value = value + 3
  WHERE key = 'likes'
`);

await db.push();
await db.pull();

const result = await db.query(`
  SELECT key, value FROM counter
`);

console.log("Client A:", result);
```

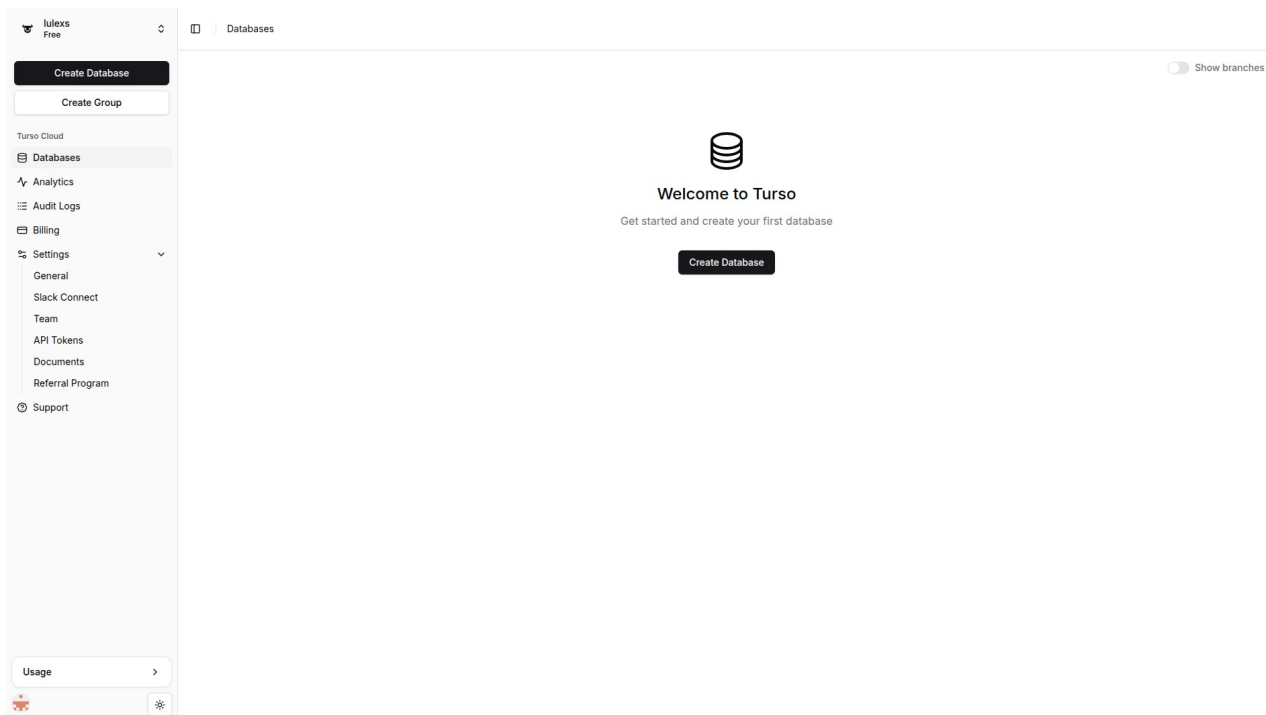
Slika 31- Operacije koje klijenti izvršavaju paralelno

Slika 30- Primer definisanja custom callback funkcije

Na slici 30 prikazan je primer funkcije koja se koristi za rezoluciju konflikata. Pretpostavimo da dva klijenta istovremeno pokušaju da urade update kolone value u tabeli counter (prikazano na slici 31). Bez custom logike na server bi stigla dve update operacije i druga bi pobedila, klasičan race condition problem. Custom funkcija detektuje izmenu i update operaciju transformiše kako ne bi došlo do gubitka podataka.

4.2 Turso cloud

Turso nudi free tier u okviru svog cloud-a, a u okviru ovog free tier-a dostupno je 100 baza podataka, 5GB skladišta, 500 miliona mesečnih čitanja, 10 miliona mesečnih upisa, 3GB sync-a i 1 dan point-in-time recovery. Osim skladišta, ovakav plan je dovoljan za većinu manjih i srednjih projekata, a više nego dovoljan za demonstracione svrhe. Prilikom kreiranja naloga za Turso cloud nije potrebno unošenje podataka vezanih za plaćanje jer se oni zahtevaju tek prilikom prelaska na neki od plaćenih planova. Kreiranje naloga moguće je preko Github ili Google naloga ili korišćenjem lične email adrese. Na slici 32 prikazana je početna stranica Turso cloud-a, a na slici 33 dijalog kojim se kreira nova baza podataka.



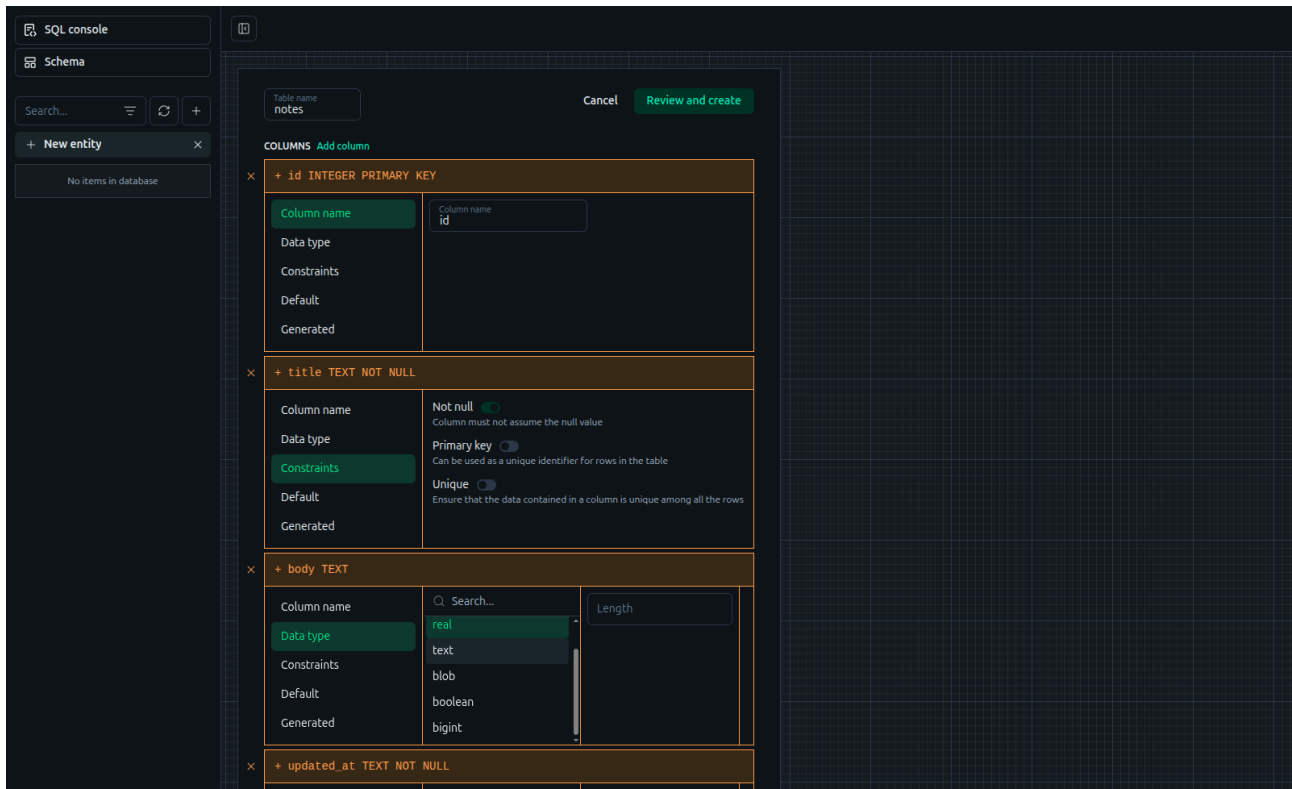
Slika 32- Početna stranica Turso cloud-a

Slika 33- Dijalog za kreiranje nove baze podataka

Kao što je više puta naglašeno, Turso u cloud-u u ovom trenutku koristi libSQL, fork SQLite-a, a plan je da se u nekom trenutku pređe na TursoDB. To znači da je Turso u potpunosti kompatibilan sa SQLite-om, tako da je bazu podataka moguće kreirati i na osnovu postojećeg SQLite fajla. Kreiranje baza podataka, branch-eva i pitr-a je jednostavno, a Turso u potpunosti ohrabruje upotrebu database-per-user obrasca, gde user mogu da budu i edge uređaji.

Povezivanje na bazu podataka i upravljanje bazom može se obaviti korišćenjem Turso cli alata ili korišćenjem Turso cloud dashboard-a. U nastavku, na slici 34 prikazano je kako je moguće kreirati tabelu korišćenjem grafičkog editora na dashboard-u.

Na slici 35 prikazane su druge funkcionalnosti koje je moguće koristiti na nivou baze podataka kroz Turso cloud dashboard – blokiranje read operacija, blokiranje write operacija, ograničavanje pristupa i slično.



Slika 34- Kreiranje tabele korišćenjem dashboard-a u cloud-u

Connect

Database URL

libsql://maindb-lulexs.aws-eu-west-1.turso.io

Create Token

Export Database

Download a copy of maindb as a SQLite file.

Download SQLite File

Configuration

Block Reads

Block all read operations on your database.

Block Writes

Block all write operations on your database.

Allow ATTACH

Allow attaching to your database to other databases.

Delete Protection

Prevent your database from being deleted.

Restrict Access

Paid plans only

Limit which clients can connect to this database by source IP address (CIDR) or AWS VPC endpoint. Leave empty to allow all traffic.

Allowed IPs (CIDR)

10.0.0.0/8

192.168.1.0/24

Allowed AWS VPC endpoint IDs

vpce-0ef5c83e127b86e9b

Slika 35- Osnovne funkcionalnosti dostupne kroz cloud dashboard

4.2.1 Embedded replicas

Za demonstraciju embedded replica funkcionalnosti, najpre će se kroz cloud dashboard, u tabeli kreiranoj na slici 34 upisati nekoliko redova, što je prikazano na slici 36.

```

▼ insert into notes
  values (1, 'Embedded replica note', 'Embedded replica will be replaced with sync', CURRENT_TIMESTAMP);

▼ insert into notes
  values (2, 'Turso is based on libSQL', 'Turso uses libSQL, which is a fork of SQLite designed for modern cloud and edge use cases.', CURRENT_TIMESTAMP);

▼ insert into notes
  values (3, 'Turso supports embedded replicas', 'Embedded replicas keep a local database file for fast local reads while writes are sent to the cloud', CURRENT_TIMESTAMP);

▼ insert into notes
  values (4, 'Embedded replicas are not fully offline-first', 'With embedded replicas, reads are local, but writes still require a connection to the cloud', CURRENT_TIMESTAMP);

▼ insert into notes
  values (5, 'Turso Sync is local-first', 'Turso Sync allows reads and writes to happen locally, then changes can be pushed to and pulled from Turso Cloud', CURRENT_TIMESTAMP);

▼ insert into notes
  values (6, 'Turso Cloud can coordinate synchronization', 'Turso Cloud can act as the remote coordination point that keeps local database files in sync', CURRENT_TIMESTAMP);

▼ insert into notes
  values (7, 'Turso is useful at the edge', 'Because data can be read from a local replica, Turso is useful for applications where low latency is important', CURRENT_TIMESTAMP);

▼ insert into notes
  values (8, 'Turso differs from Postgres', 'A traditional Postgres app usually depends on a remote database server, while Turso can support local replicas', CURRENT_TIMESTAMP);

▼ insert into notes
  values (9, 'SQLite developer experience', 'Turso keeps much of the simplicity of SQLite while adding cloud features such as replication, authentication, and scaling', CURRENT_TIMESTAMP);

```

Slika 36- Insert operacije izvršene iz cloud konzole

```
import libsql

conn = libsql.connect(
    "local-replica.db",
    sync_url="libsql://maindb-lulexs.aws-eu-west-1.turso.io",
    auth_token="eyJhbGciOiJIJZERTQSI6InR5cCI6IkpvcXV9.eyJhIjoicnciLCJleHAiOjE3ODMyNTE3NzMsImIhdCI6MTc4MzE2NTM3MywiaWQiOiIwMTlmmj"
)

conn.sync()

conn.execute(
    "INSERT INTO notes VALUES (?, ?, ?, ?)",
    ("10", "Offline demo", "This app reads locally but writes through Turso Cloud.", "CURRENT_TIMESTAMP")
)

conn.commit()

rows = conn.execute("SELECT count(*) FROM notes").fetchall()
print(rows)
```

Slika 37- Korišćenje embedded replike u Python-u

```
(venv) lule@lule-Asus-Vivobook:~/Desktop/DBMS/P3/impl$ python embedded_replicas.py
[(10,)]
(venv) lule@lule-Asus-Vivobook:~/Desktop/DBMS/P3/impl$ python embedded_replicas.py
```

Slika 38- Rezultat izvršenja koda sa slike 37

Na slici 37 prikazan je osnovni primer kako se koristi embedded replika funkcionalnost. Nakon uspostavljanja konekcije sa Turso cloudom, poziva se `sync`, čime se lokalna replika popunjava podacima sa servera. Nakon toga se vrši upis podataka, a zbog načina na koji embedded replike funkcionišu, taj podataka se odmah prosleđuje primary bazi u cloud-u, nakon čega se vrši `commit`. Izvršenje upita kojim se prikazuje broj redova rezultovaće brojem 10, što je 9 upisanih redova sa slike 36 i upis koji se desio u kodu. Na ovaj način se potvrđuje *read your writes* garancija konzistencije koju Turso obezbeđuje prilikom korišćenja embedded replika.

Interesantno je ispitati ponašanje embedded replika kada se upis vrši u lokalni fajl bez primary baze u cloudu. Najpre je izvršen kod prikazan na slici 39, čime je podataka upisan u lokalni fajl. Nakon toga izvršen je kod sa slike 40 gde je izvršen sync, a potom pribavljen podatak koji je prethodno upisan u lokalni fajl. Može se primetiti da se tom podatku moglo pristupiti lokalno, ali se podatak nije nalazio u cloudu, što je prikazano na slikama 41 i 42. Time se pokazuje da kod embedded replika, nije out-of-the-box podržana dvosmerna sinhronizacija te je za takve slučajeve potrebno razviti custom logiku kojom bi se lokalni upisi sinhronizovali sa cloud-om.

```
import libsql

conn = libsql.connect(
    "local-replica.db",
)

conn.execute(
    "INSERT INTO notes VALUES (?, ?, ?, CURRENT_TIMESTAMP)",
    ("12", "Local write", "This write is local with no primary in cloud")
)

conn.commit()
```

Slika 39- Upis u lokalni fajl, bez upisa u primary server

```
import libsql

conn = libsql.connect(
    "local-replica.db",
    sync_url="libsql://maindb-lulexs.aws-eu-west-1.turso.io",
    auth_token="eyJhbGciOiJIJZERTQSIInR5cCI6IkpXVCJ9.eyJhIjoicnciLCJleHAiOjE3ODMyNTE3NzMsImhhdCI6MTc4MzE2NTM3MywiaWQiOiIwMTlmMj"
)

conn.sync()

rows = conn.execute("SELECT * FROM notes where id = 12").fetchall()
print(rows)
```

Slika 40- Sinhronizacija sa cloud-om nakon izvršenja koda sa slike 39

```
(venv) lule@lule-Asus-Vivobook:~/Desktop/DBMS/P3/impl$ python embedded_replicas.py
[(12, 'Local write', 'This write is local with no primary in cloud', '2026-07-04 12:05:02')]
```

Slika 41- Rezultat izvršenja koda sa slike 40

☐	id	title	body	updated_at
☐	1	Embedded replica note	Embedded replica will be rep...	2026-07-04 11:37:29
☐	2	Turso is based on libSQL	Turso uses libSQL, which is ...	2026-07-04 11:39:23
☐	3	Turso supports embedded repl...	Embedded replicas keep a loc...	2026-07-04 11:39:24
☐	4	Embedded replicas are not fu...	With embedded replicas, read...	2026-07-04 11:39:25
☐	5	Turso Sync is local-first	Turso Sync allows reads and ...	2026-07-04 11:39:26
☐	6	Turso Cloud can coordinate s...	Turso Cloud can act as the r...	2026-07-04 11:39:27
☐	7	Turso is useful at the edge	Because data can be read fro...	2026-07-04 11:39:27
☐	8	Turso differs from Postgres	A traditional Postgres app u...	2026-07-04 11:39:28
☐	9	SQLite developer experience	Turso keeps much of the simp...	2026-07-04 11:39:29
☐	10	Offline demo	This app reads locally but w...	CURRENT_TIMESTAMP

Slika 42- Podaci koji se nalaze u cloud-u nakon izvršenja koda sa slike 40

```
conn = libsql.connect(
    "local-replica.db",
    sync_url="libsql://maindb-lulexs.aws-eu-west-1.turso.io",
    auth_token="eyJhbGciOiJIJZERTQSIInR5cCI6IkpXVCJ9.eyJhIjoicmciLCJleHAiOjE3ODMyNTE3NzMsImldCI6MTc4MzE2NTM3MywiaWQiOiIwMTlmMj",
    sync_interval=60,
    offline=True
)
```

Slika 43- Podešavanje konekcija za upotrebu periodičnog sync-a

Na slici 43 prikazano je kako se embedded replica može koristiti u „offline“ režimu. Kada je ovo podešavanje uključeno svaki upis se dešava nad lokalnim fajlom, a kako je u prethodnom pasusu prikazana nemogućnost dvosmerne replikacije, to znači da se u ovom režimu podaci upisuju samo u lokalni fajl. Pored toga, podešavanjem parametra konekcije *sync_interval* moguće je automatizovati

proces sinhronizacije sa primary serverom tako što se sync ponavlja bez potrebe za eksplicitnim pozivom sync-a nad konekcijom, periodično.

4.2.2 Branching

Grane mogu da se kreiraju korišćenjem Turso CLI alata, ili korišćenjem cloud dashboard-a. Na slici 44 prikazan je postupak kreiranja grane od aktuelnog stanja baze podataka. Pregledom podataka primećuje se da su u bazi svi podaci iz glavne baze podataka prisutni, kao i da modifikacija podataka u bilo kojoj grani ne utiče na drugu granu. Na slici 45 prikazan je sadržaj grane nakon upisa jednog reda, dok je na slici 46 prikazan sadržaj glavne baze nakon upisa jednog reda. Kao što se sa slika može primetiti, obe grane sadrže po jedan red koji druga grana ne sadrži. Grane (kao između ostalog i sama baza podataka) se vrlo jednostavno briše.

4.2.3 Point-in-time recovery

Najjednostavniji način za korišćenje point-in-time recovery mehanizma je korišćenjem grana. Na slici 47 prikazano je kreiranje grane na osnovu stanja baze podataka pre sat vremena, a za finiju kontrolu pitr vremena može se koristiti cli alat i komanda u nastavku:

```
turso db create my-new-database --from-db my-existing-database --timestamp 2024-01-01T00:00:00Z.
```

Na slici 48 prikazano je stanje novokreirane grane, gde se primećuje da podaci upisani u toku prikazivanja grananja u prethodnoj sekciji nisu uključeni.

The screenshot shows the Turso Cloud dashboard interface. At the top, there's a navigation bar with tabs: Databases, maindb, Overview, Branches, Analytics, Top Queries, and Edit Data. The 'Branches' tab is active, showing a 'Create a branch' section with two buttons: 'Create From Now' and 'Create From Point-in-Time'. Below this, there's a list of branches, currently empty. A modal window titled 'Create Branch' is open, showing a table with 12 records. The table has columns: id, title, body, and updated_at. The records are as follows:

id	title	body	updated_at
1	Embedded replica note	Embedded replica will ...	2026-07-04 11:37:29
2	Turso is based on libS...	Turso uses libSQL, whi...	2026-07-04 11:39:23
3	Turso supports embedde...	Embedded replicas keep...	2026-07-04 11:39:24
4	Embedded replicas are ...	With embedded replicas...	2026-07-04 11:39:25
5	Turso Sync is local-fi...	Turso Sync allows read...	2026-07-04 11:39:26
6	Turso Cloud can coordi...	Turso Cloud can act as...	2026-07-04 11:39:27
7	Turso is useful at the...	Because data can be re...	2026-07-04 11:39:27
8	Turso differs from Pos...	A traditional Postgres...	2026-07-04 11:39:28
9	SQLite developer exper...	Turso keeps ...	2026-07-04 11:39:29
10	Offline demo	This app reads locally...	CURRENT_TIMESTAMP
11	Offline demo	This app reads locally...	CURRENT_TIMESTAMP
12	Branch write	This write happend on ...	2026-07-04 15:35:00

Slika 45- Sadržaj grane nakon upisa jednog reda

Databases > maindb > Overview Branches Analytics Top Queries Edit Data

SQL console

Schema

Search...

notes

DATA

STRUCTURE

Filters

Sort

Columns

Add record

116ms

1-12 of 12

id integer	title text	body text	updated_at text	
☐	1	Embedded replica note	Embedded replica will ..	2026-07-04 11:37:29
☐	2	Turso is based on libS...	Turso uses libSQL, whi...	2026-07-04 11:39:23
☐	3	Turso supports embedde...	Embedded replicas keep...	2026-07-04 11:39:24
☐	4	Embedded replicas are ..	With embedded replicas...	2026-07-04 11:39:25
☐	5	Turso Sync is local-fi...	Turso Sync allows read...	2026-07-04 11:39:26
☐	6	Turso Cloud can coordi...	Turso Cloud can act as...	2026-07-04 11:39:27
☐	7	Turso is useful at the...	Because data can be re...	2026-07-04 11:39:27
☐	8	Turso differs from Pos...	A traditional Postgres...	2026-07-04 11:39:28
☐	9	SQLite developer exper...	Turso keeps much of th...	2026-07-04 11:39:29
☐	10	Offline demo	This app reads locally...	CURRENT_TIMESTAMP
☐	11	Offline demo	This app reads locally...	CURRENT_TIMESTAMP
☐	12	Main db write	This write happened on...	CURRENT_TIMESTAMP

dirizle studio

Slika 46- Sadržaj glavne baze nakon upisa jednog reda

Databases > maindb > Overview Branches Analytics Top Queries Edit Data

Create a branch

Create a new branch from maindb, either from its current state or from a point in time.

Create From Now

Create From Point-in-Time

Branches

Databases that were branched from maindb

No branches

Create Database from Backup

Create a new database with the same schema and data as this one at a specific point in time.

Database Name

pltr-example

Restore from

1 Hour Ago

Create Database

Slika 47- Kreiranje grane na osnovu tačke u vremenu

SQL console

Schema

Search...

notes

Settings

Share

Powered by drizzle studio

	DATA	STRUCTURE	<	>	⌂ Filters	⬇ Sort	⌵ Columns	+ Add record	108ms	< 1-11 of 11 >	↺	⋮
☐	id integer				title text		body text		updated_at text			+
☐	1				Embedded replica note		Embedded replica will _		2026-07-04 11:37:29			
☐	2				Turso is based on libS_		Turso uses libSQL, whi_		2026-07-04 11:39:23			
☐	3				Turso supports embedde_		Embedded replicas keep_		2026-07-04 11:39:24			
☐	4				Embedded replicas are _		With embedded replicas_		2026-07-04 11:39:25			
☐	5				Turso Sync is local-fi_		Turso Sync allows read_		2026-07-04 11:39:26			
☐	6				Turso Cloud can coordi_		Turso Cloud can act as_		2026-07-04 11:39:27			
☐	7				Turso is useful at the_		Because data can be re_		2026-07-04 11:39:27			
☐	8				Turso differs from Pos_		A traditional Postgres_		2026-07-04 11:39:28			
☐	9				SQLite developer exper_		Turso keeps much of th_		2026-07-04 11:39:29			
☐	10				Offline demo		This app reads locally_		CURRENT_TIMESTAMP			
☐	11				Offline demo		This app reads locally_		CURRENT_TIMESTAMP			

Slika 48- Sadržaj grane kreirane uz pomoć PITR

5 Literatura

[1] “SQLite Documentation.” *Sqlite.org*, sqlite.org/docs.html.

[2] *Turso.tech*, 2026, docs.turso.tech/libsql.

[3] tursodatabase. “Libsql/Libsql-Sqlite3/Doc/Libsql_extensions.md at Main · Tursodatabase/Libsql.” *GitHub*, 2025, github.com/tursodatabase/libsql/blob/main/libsql-sqlite3/doc/libsql_extensions.md. Accessed 17 June 2026.

[4] Enberg, Pekka. “Beyond the Single-Writer Limitation with Turso’s Concurrent Writes.” *Turso.tech*, 6 Oct. 2025, turso.tech/blog/beyond-the-single-writer-limitation-with-tursos-concurrent-writes.

[5] Sivukhin, Nikita. “Introducing Databases Anywhere with Turso Sync.” *Turso.tech*, Turso, 8 Oct. 2025, turso.tech/blog/introducing-databases-anywhere-with-turso-sync. Accessed 20 June 2026.

[6] Mintlify. “Turso.” *Turso.tech*, Turso, 2026, docs.turso.tech/turso-cloud. Accessed 21 June 2026.

[7] Ismat Samadov. “Database Branching in 2026: How Neon, Supabase, Xata, and Turso Killed the Staging Environment.” *Birjob.com*, BirJob, 17 May 2026, www.birjob.com/blog/database-branching-2026. Accessed 21 June 2026.

[8] Costa, Glauber. “How Does Turso Cloud Keep Your Data Durable and Safe?” *Turso.tech*, Turso, 20 May 2025, turso.tech/blog/how-does-the-turso-cloud-keep-your-data-durable-and-safe. Accessed 21 June 2026.